Hireonomous

AI Hiring Automation Platform

Technical Documentation

Version 1.0.0 · September 2026

Architecture, modules, API reference, data model, configuration and deployment

Contents

1. Overview 4

1.1 Key capabilities . . . 4

1.2 Repository layout . . . 4

2. System architecture 5

2.1 Technology stack . . . 5

2.2 Backend layering . . . 6

3. Hiring pipeline 7

3.1 Candidate status lifecycle . . . 7

4. Functional modules 7

4.1 Resume screening . . . 7

4.2 Jobs and the AI Hiring Assistant . . . 8

4.3 AI phone screening (Bolna) . . . 8

4.4 Email . . . 8

5. AI interview engine 9

5.1 Interview lifecycle . . . 9

5.2 Question plan . . . 9

5.3 Turn processing and follow-ups . . . 9

5.4 Evaluation . . . 9

5.5 Integrity analysis . . . 10

6. Security and multi-tenancy 11

6.1 Roles . . . 11

6.2 Authentication boundaries . . . 11

6.3 Onboarding flow . . . 11

7. Billing 11

8. Data model 13

8.1 Interview document . . . 13

9. API reference 14

Authentication - /api/auth . . . 14

Resumes and candidates . . . 14

Jobs and AI Hiring Assistant . . . 14

Calling and email . . . 14

AI interviews - recruiter . . . 15
AI interviews - candidate (token in path, no JWT) . . . 15
Analytics - /analytics . . . 15
Billing - /api/billing . . . 16
Super admin - /api/superadmin . . . 16
Health . . . 16

10. Frontend **17**

10.1 Candidate interview room . . . 17

11. Configuration **17**

12. Deployment **19**

13. Known limitations and recommendations **19**

1. Overview

Hireonomous is a multi-tenant SaaS platform that automates the front half of a hiring funnel. A company posts a job, uploads resumes, and the platform scores each resume against the job description, emails shortlisted candidates, screens them over an AI phone call, runs one or two rounds of AI-conducted video interviews, and hands the recruiter a scored report with integrity flags. The recruiter makes the final select / reject decision, which triggers the matching candidate email.

Every AI output in the system is **advisory**. The AI recommendation and the recruiter decision are stored in separate fields, integrity analysis produces flags rather than verdicts, and candidate emails for selection or rejection are sent only on an explicit recruiter action.

1.1 Key capabilities

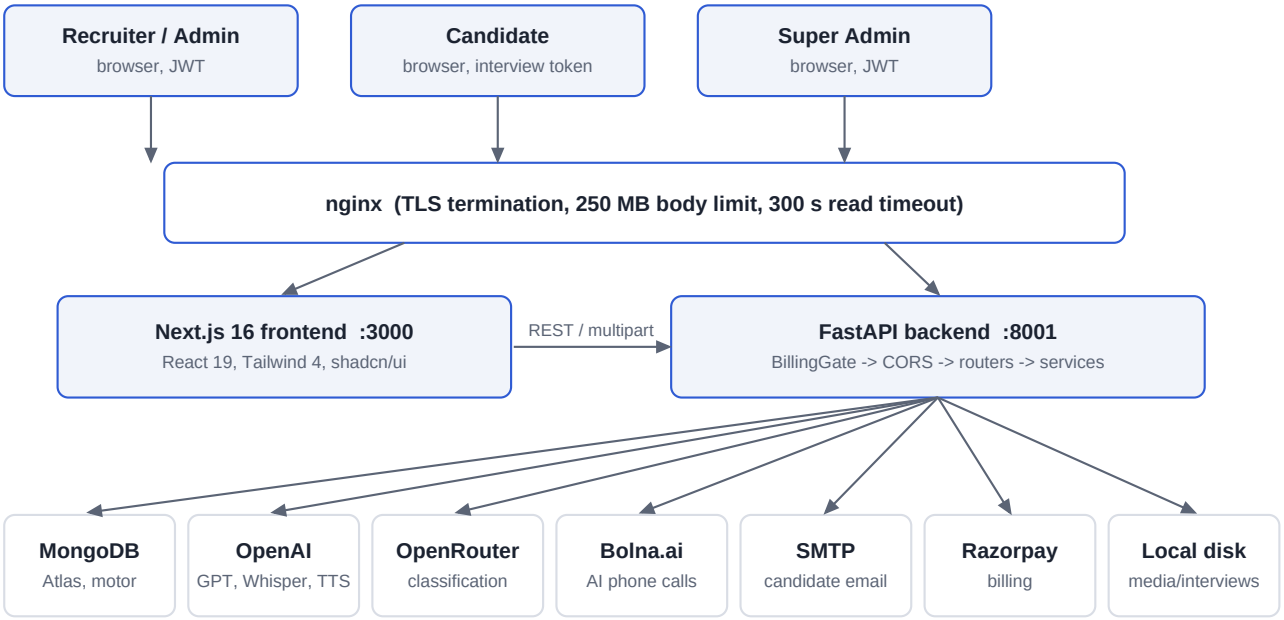
- **Resume screening** - PDF / DOCX parsing (PyMuPDF, python-docx), bulk upload or Google Drive folder import, a single GPT-4o-mini call that extracts candidate fields and scores fit against the JD (0-100).
- **AI phone screening** - outbound calls through Bolna.ai with a per-job prompt built from the AI Hiring Assistant configuration; webhooks write the transcript, interest and scores back onto the candidate.
- **L1 AI interview** - a tokenised candidate link opens a browser interview; questions are spoken with TTS, answers are recorded as video/audio, transcribed with Whisper, scored per answer, and up to 3 adaptive follow-ups are asked.
- **L2 technical interview** - a deeper second round (`ai_l2_technical`) with stricter integrity thresholds.
- **Integrity analysis** - tab switching, paste, fullscreen exits, answer latency, multi-IP / multi-device access, and an LLM pass for read-aloud or AI-generated answers.
- **Analytics** - dashboard, funnel, per-role breakdown, trend, report and Excel/CSV export.
- **Multi-tenancy and billing** - organisations, role-based users, a super-admin console with approvals and "view as organisation", a 7-day trial and Razorpay recharge enforced server-side.

1.2 Repository layout

```
hirenomous/
|-- backend/                FastAPI service (Python 3.11)
|   |-- app/main.py         app factory, middleware, router registration
|   |-- app/core/           config, database, JWT auth, interview-token auth, billing gate
|   |-- app/api/deps.py     auth dependencies, org scoping, super-admin guard
|   |-- app/api/routes/     13 routers (thin HTTP layer)
|   |-- app/services/       business logic (interview, resume, bolna, billing, email, ...)
|   |-- app/models/         Pydantic models (candidate, interview, user, billing, job_ai_config)
|   `-- scripts/            one-off migrations and maintenance
|-- frontend/               Next.js 16 app router (TypeScript)
|   |-- app/(dashboard)/    recruiter workspace pages
|   |-- app/interview/[token] candidate interview room
|   |-- app/superadmin/     platform admin console
|   |-- components/, lib/   UI kit, API client, auth context, billing helpers
`-- DEPLOYMENT.md           AWS EC2 runbook
```

2. System architecture

The system is two processes behind nginx: a Next.js frontend and a FastAPI backend. All persistent state lives in MongoDB Atlas except interview recordings, which are written to the backend host's disk. External AI and communication providers are called directly over HTTPS with `httpx`; no vendor SDKs are used except Twilio (legacy).



2.1 Technology stack

Layer	Technology	Notes
Frontend	Next.js 16.3, React 19, TypeScript 5.7	App router; route groups for dashboard, superadmin, candidate interview
UI	Tailwind CSS 4, shadcn/ui, Base UI, lucide-react, Recharts 3	Charts on dashboard and analytics
Backend	FastAPI, Uvicorn, Pydantic v2 / pydantic-settings	Async throughout; blocking work pushed to a threadpool
Database	MongoDB Atlas via Motor 3.1 (PyMongo 4.3)	Schemaless documents keyed by a UUID `id` field
LLM	OpenAI GPT-4o-mini; OpenRouter (configurable model)	Resume scoring, question plans, answer analysis, evaluation
Speech	OpenAI Whisper (STT), OpenAI TTS; faster-whisper available	Interview questions spoken, answers transcribed
Telephony	Bolna.ai (active); Twilio + Vapi (legacy, routers disabled)	Outbound AI screening calls
Documents	PyMuPDF, python-docx, openpyxl, pandas	Resume/JD parsing, Excel export
Auth	PyJWT (HS256), passlib + bcrypt	7-day recruiter tokens; HMAC-hashed interview tokens
Payments	Razorpay REST API	Orders, signature verification, webhooks
Hosting	AWS EC2 (Ubuntu 24.04), nginx, certbot, systemd	See section 11

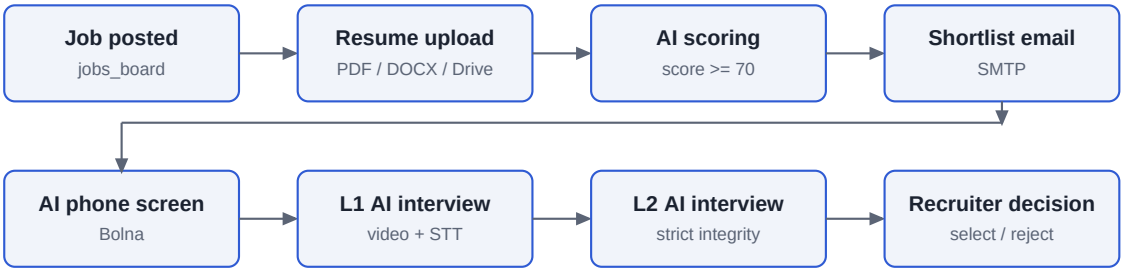
2.2 Backend layering

Routers in `app/api/routes` are thin: they resolve the caller (JWT or interview token), extract the organisation id, and delegate to a service. Services in `app/services` own all business rules, database access and provider calls. Cross-cutting concerns live in `app/core`.

Request stage	Component	Responsibility
1	CORSMiddleware (outermost)	Adds CORS headers, including on 402 responses from the billing gate
2	BillingGateMiddleware	Decodes the bearer JWT; returns 402 `billing_required` if the org's trial and paid period have both ended
3	Router + dependencies	`get_current_active_user`, `get_context_organization_id`, `require_super_admin`, or `resolve_interview_by_token`
4	Service	Business logic, MongoDB reads/writes, AI/telephony/email calls, audit events
Startup	lifespan hooks	Connect to MongoDB (5 s timeout), create interview, hiring-event and payment indexes

3. Hiring pipeline

A candidate moves through the pipeline below. Each stage writes to the candidate document's `status` field (L2 progress uses a separate `l2_status` so that L1 data is preserved).



3.1 Candidate status lifecycle

Status	Set by	Meaning
`uploaded`	Resume upload	Parsed, not yet scored or skip_ai used
`shortlisted / rejected`	Resume scoring	Score at or above / below SHORTLIST_THRESHOLD (default 70)
`calling`	Bolna call start / webhook	Screening call in progress
`interested / not_interested`	Bolna webhook + LLM classification	Outcome of the phone screen
`callback_required`	Bolna webhook	No answer, busy, or candidate asked to be called later
`interview`	Interview created	L1 interview invited; `latest_interview_id` set
`interview_completed`	Candidate completes L1	Evaluation runs in the background
`selected / rejected`	Recruiter decision	Triggers selection or regret email
`l2_status: interview / interview_completed`	L2 interview	Tracked separately; `l2_interview_id` set

AI interviews can only be created for candidates whose status is `interested`, `interview`, `interview_completed` or `selected`. Only one non-terminal interview may exist per candidate and job; a second create call returns the existing interview.

4. Functional modules

4.1 Resume screening

`POST /upload-resume` accepts a resume file plus either typed JD text or a JD file. The file is written to `temp_resumes/`, text is extracted in a threadpool, and `ResumeService.parse_and_score_resume_with_gpt` makes a single OpenAI call that returns name, contact details, skills, missing skills, experience summary and a 0-100 fit score. Combining parse and score into one call roughly halves latency and cost per resume.

- If the AI call fails, a regex/keyword fallback parser still extracts name, email, phone and known skills.
- `clean_json_response` tolerates markdown fences, prefixes, trailing commas and control characters in model output.

- Resumes can also be imported from a public Google Drive folder (`/list-drive-folder-files`, `/upload-resume-from-drive`) or added manually.
- The shared `httpx.AsyncClient` is pooled (15 keep-alive / 25 max connections) for bulk uploads.

4.2 Jobs and the AI Hiring Assistant

Jobs live in `jobs_board`. Each job may have a `job_ai_config` document that sets language, tone, voice, screening mode and an ordered list of screening questions. The assistant can generate questions with the LLM, preview the assembled call prompt, and simulate a call. `PromptEngine` turns this configuration into the system prompt sent to Bolna.

4.3 AI phone screening (Bolna)

- `POST /bolna/call-candidate/{id}` or `/bolna/call-shortlisted` starts outbound calls. Numbers without a country code default to +91.
- Bolna posts call results to `/bolna/webhook`; the agent's mid-call callback hits `/bolna/callback`, which requires `BOLNA_CALLBACK_API_TOKEN`.
- `BolnaService.process_webhook_payload` maps call status, stores the transcript and recording URL, and uses `extract_metrics_with_llm` to derive interest, communication, technical and confidence scores.
- `/bolna/sync-call/{id}` pulls call details on demand if a webhook was missed.

4.4 Email

`EmailService` sends over SMTP (TLS on 587). Candidate-facing mail is branded per organisation: the From name is "<Company> Hiring Team" and Reply-To is the organisation's `reply_to` or `contact_email`. Templates cover shortlist, interview invite, selection, rejection and password reset.

5. AI interview engine

The AI interview is the core of the product. Logic lives in `InterviewService` (orchestration), `interview_prompt_engine` (LLM prompts), `interview_integrity` (proctoring) and `InterviewStorage` (media).

5.1 Interview lifecycle

Step	Endpoint	What happens
Create	`POST /interviews`	Checks candidate eligibility, generates a question plan, creates a random token (stores only its HMAC-SHA256 hash), snapshots job and resume context, optionally emails the invite
Open link	`GET /interview-session/{token}`	Candidate sees job title, question count and whether email verification is required
Verify	`POST /interview-session/{token}/verify`	Candidate confirms the email they were shortlisted with (max 6 attempts)
Start	`POST /interview-session/{token}/start`	Status to in_progress; first question returned with TTS audio
Answer	`POST /interview-session/{token}/turn`	Multipart video/audio or typed text plus browser telemetry; transcribe, score, decide follow-up, return next question
Complete	`POST /interview-session/{token}/complete`	Status to completed; evaluation and integrity analysis run as a background task
Review	`GET /interviews/{id}/report`	Scores, strengths, gaps, summary, integrity flags, per-answer recordings
Decide	`POST /interviews/{id}/decision`	select / reject / needs_review; select and reject send candidate email

5.2 Question plan

`generate_question_plan` produces a fixed, reproducible plan of `INTERVIEW_QUESTION_COUNT` (default 8) questions. Every L1 interview opens with the same introduction question; L2 opens with an advanced-round welcome. Remaining questions are typed (technical, resume_specific, problem_solving, scenario, behavioral, closing) and tagged with target skills. A deterministic fallback plan is used if the LLM call fails.

5.3 Turn processing and follow-ups

- Video is saved for recruiter review; a smaller audio clip is preferred for STT, falling back to the video file.
- Each answer is scored 0-100 by `analyze_answer`, which also decides whether one follow-up would help.
- At most one follow-up per base question, never a follow-up on a follow-up, and at most `INTERVIEW_MAX_FOLLOWUPS` (3) per interview.
- Navigation walks only the base plan; follow-ups are appended to `question_plan` for the record.
- Duplicate submissions (network retries) are detected and advance the interview without re-recording.

5.4 Evaluation

After completion, `evaluate_interview` scores six dimensions against the job-anchored rubric (`interview-rubric-v1`). The overall score is computed deterministically from fixed weights, not by the model:

Dimension	Weight
Technical knowledge	28%
Problem solving	22%
Role-specific skills	20%
Experience	12%
Communication	10%
Answer relevance	8%

Overall score	Recommendation
82 and above	strong_match
65 - 81	match
45 - 64	weak_match
below 45	no_match

If the candidate gave no substantive answers (under 25 characters in total), the LLM is not called; the interview is marked `needs_review` with an honest "not evaluable" summary so the model cannot invent a score from the resume. All prompts include fairness rules that forbid using age, gender, ethnicity, accent, name, university prestige, employment gaps and similar attributes.

5.5 Integrity analysis

Integrity analysis is advisory and uses signals available without extra ML infrastructure. Each flag has a severity; severity weights (low 8, medium 22, high 45) sum into a concern score capped at 100, which maps to a level of `clean`, `review` or `high_risk`.

Flag	Signal	L1 severity	L2 severity
`multi_ip / multi_device`	Session opened from several networks or browsers	medium - high	medium - high
`email_verify_failed`	Repeated wrong email at verification	medium (high at 4+)	same
`tab_switching`	Focus lost while answering	from 4 s away	from 2 s away; high at 10 s or 2 events
`paste`	Text pasted into an answer	medium	high
`fullscreen_exit`	Left fullscreen	low	medium
`answer_latency`	12 s+ before answering	low - medium	up to high
`typed_answers`	Typed instead of spoken	low	medium
`ai_generated_text`	LLM transcript pass	medium - high	medium - high
`follow_up_handling / consistency`	LLM transcript pass	medium / low	medium / low

L2 levels are zero-tolerance: any single high flag, two serious flags, or a score of 40+ is `high_risk` (L1 needs 55+, or one high plus two serious). Face match, gaze tracking and voice fingerprinting are deferred to a later version.

6. Security and multi-tenancy

6.1 Roles

Role	Scope
`SUPER_ADMIN`	Platform operator. Approves access requests, creates companies, grants free access, sees platform stats. Never billing-gated. Can act inside any org with the `X-View-As-Org` header.
`ORGANIZATION_ADMIN`	Created when a company's access request is approved. Full access to its own organisation.
`RECRUITER / HIRING_MANAGER`	Organisation-scoped users.

6.2 Authentication boundaries

- **Recruiter JWT** - HS256, 7-day expiry, claims `sub`, `role`, `org_id`. Passwords hashed with bcrypt; verification runs in a threadpool.
- **Candidate interview token** - a separate boundary. 32 bytes from `secrets.token_urlsafe`, returned once to the recruiter and embedded only in the link. The database stores `HMAC-SHA256(INTERVIEW_TOKEN_SECRET or SECRET_KEY, token)` with a unique sparse index. Links expire after `INTERVIEW_TOKEN_TTL_HOURS` (48 h default).
- **Tenant isolation** - every recruiter query filters by `organization_id` taken from the authenticated user (via `get_context_organization_id`), never from the request body.
- **Audit trail** - `AuditService.record` writes interview creation, uploads, completion, evaluation, integrity and recruiter decisions to `hiring_events`.
- **PII minimisation** - `safe_resume_context` passes only skills, experience and summary to interview prompts; recordings are stored as file references, never embedded in documents.

6.3 Onboarding flow

A company submits `/request-access` with a password (stored hashed on the request). A super admin approves it, which creates the organisation with `trial_ends_at = now + TRIAL_DAYS` and an ORGANIZATION_ADMIN user using the stored hash. Password reset uses `/api/auth/forgot-password` and `/reset-password` by email.

7. Billing

`billing_service.compute_access` is the single rule for whether an organisation can use the app, shared by the middleware and the status API so they cannot disagree:

Condition	Status	Access
No trial_ends_at and no paid_until (created before billing)	legacy	Yes
paid_until in the future	active	Yes
trial_ends_at in the future	trial	Yes (days_left shown)
Otherwise, or organisation missing	expired	No - API returns 402

- **Recharge** - `/api/billing/create-order` creates a Razorpay order (default plan: INR 2,999 for 30 days); `/verify` checks the HMAC signature and extends `paid_until` from whichever is later, now or the current paid_until.
- **Webhook backstop** - `/api/billing/webhook` applies payments whose verify call never arrived (e.g. tab closed). A unique index on `razorpay_order_id` makes application idempotent.

- **Without Razorpay keys** - the trial gate still enforces; the paywall shows "contact your account manager" and a super admin can use Grant 30d.
- **Fail-open** - if the billing lookup itself errors, the request is allowed through and the error is logged.

8. Data model

MongoDB database `voiceai` (configurable). Documents use a UUID string `id` as the application key.

Collection	Purpose	Key fields / indexes
`organizations`	Tenants	id, name, status, contact_email, reply_to, trial_ends_at, paid_until
`users`	Login accounts	id, email, hashed_password, role, organization_id, status
`access_requests`	Company sign-up requests	company, email, password hash, status (pending / approved / rejected)
`jobs_board`	Job postings	id, organization_id, title, description, skills
`job_ai_config`	AI Hiring Assistant config per job	job_id, language, tone, voice, screening_questions[], is_active
`candidates`	Central candidate record	resume fields, resume_score, status, call data (bolna_call_id, transcript, scores), latest_interview_id, l2_interview_id, l2_status
`interviews`	One document per AI interview	question_plan[], answers[], transcript[], scores, recommendation, ai_report, integrity, recruiter_decision, snapshot. Indexes: (organization_id, status), candidate_id, job_id, token_hash unique sparse, id unique
`hiring_events`	Audit trail	event_type, actor_type, actor_id, candidate/job/interview ids, payload. Indexes: (organization_id, created_at), interview_id
`payments`	Razorpay orders and payments	organization_id, razorpay_order_id (unique), plan, amount, status
`calls`, `call_snapshots`	Legacy call records	Used by the Twilio / simulation paths

8.1 Interview document

```
{
  id, organization_id, candidate_id, job_id,
  interview_type: "ai_technical" | "ai_l2_technical",
  status: invited | scheduled | in_progress | completed | cancelled | expired | failed,
  evaluation_status: pending | evaluated | needs_review,
  token_hash, token_expires_at, email_verified, verification_attempts,
  question_plan: [{ id, text, question_type, sequence, target_skills, is_followup, parent_question_id }],
  answers: [{ question_id, answer_text, audio_ref, video_ref, score, feedback, duration_seconds,
              typed_only, focus_lost_count, focus_lost_ms, paste_count, fullscreen_exits,
              time_to_first_answer_ms }],
  transcript: [{ role: "ai" | "candidate", text, ts }],
  progress: { plan_index, followups_used, pending_question },
  scores: { overall, technical_knowledge, problem_solving, communication, role_specific,
            experience, answer_relevance },
  recommendation, ai_report: { strengths, areas_to_improve, summary, llm_recommendation },
  integrity: { score, level, flags: [{ code, severity, title, detail }], signals, analyzed_at },
  recruiter_decision, recruiter_feedback, recruiter_id, decided_at,
  session_meta: [{ ip, ua, at, event }],
  snapshot: { job_context, resume_context, model, rubric_version, job_title, generated_at }
}
```

9. API reference

All recruiter endpoints require `Authorization: Bearer <JWT>` and are scoped to the caller's organisation. Interactive OpenAPI docs are served at `/docs`.

Authentication - /api/auth

Method	Path	Description
POST	`/api/auth/login`	Email + password; returns JWT and user
POST	`/api/auth/request-access`	Company sign-up request
POST	`/api/auth/forgot-password`	Email a reset link
POST	`/api/auth/reset-password`	Set a new password
GET	`/api/auth/me`	Current user with organisation name

Resumes and candidates

Method	Path	Description
POST	`/upload-resume`	Upload one resume + JD; parse and score
POST	`/upload-resume-from-drive`	Import one file from Google Drive
POST	`/list-drive-folder-files`	List resumes in a Drive folder
POST	`/add-manual`	Add a candidate without a resume
GET	`/candidates`	List candidates
GET	`/shortlisted`	Candidates at or above the threshold
PUT	`/candidates/{id}/status`	Change status
DELETE	`/candidates/{id}, /candidates/bulk`	Delete one or many
GET	`/final, /export/candidates, /export/calls`	Final list and Excel exports
POST	`/reset-session`	Clear the org's working session

Jobs and AI Hiring Assistant

Method	Path	Description
POST/GET	`/job, /jobs, /job/{id}`	Create, list, read
PUT/DELETE	`/job/{id}`	Update, delete
GET/PUT	`/api/ai-recruiter/{job_id}`	Read / save screening config
POST	`/api/ai-recruiter/{job_id}/generate-questions`	LLM-generated screening questions
GET	`/api/ai-recruiter/{job_id}/preview`	Preview the call prompt
POST	`/api/ai-recruiter/{job_id}/simulate`	Simulate a screening call

Calling and email

Method	Path	Description
POST	`/bolna/call-candidate/{id}`	Call one candidate

Method	Path	Description
POST	`/bolna/call-shortlisted`	Call all shortlisted candidates
POST	`/bolna/stop-call/{id}`	Stop a call
POST	`/bolna/webhook`	Bolna call results (no auth)
POST	`/bolna/callback`	Agent callback (token-authenticated)
GET	`/bolna/sync-call/{id}`	Pull call details from Bolna
POST	`/bolna/save-call-result/{id}`	Save a call result manually
POST	`/email/send-shortlisted`	Email all shortlisted candidates

AI interviews - recruiter

Method	Path	Description
POST	`/interviews`	Create (and optionally invite)
POST	`/interviews/bulk-invite`	Invite every eligible candidate, optionally per job
GET	`/interviews`	Filter by job, status, recommendation, interview_type
GET	`/interviews/{id}`	Interview detail
POST	`/interviews/{id}/send-invite`	Resend (rotates the token)
POST	`/interviews/{id}/cancel`	Cancel
GET	`/interviews/{id}/report`	Evaluation report
POST	`/interviews/{id}/reevaluate`	Retry evaluation
POST	`/interviews/{id}/integrity`	Re-run integrity analysis
GET	`/interviews/{id}/transcript`	Transcript
GET	`/interviews/{id}/recording/{n}`	Stream answer n's recording
POST	`/interviews/{id}/decision`	select / reject / needs_review

AI interviews - candidate (token in path, no JWT)

Method	Path	Description
GET	`/interview-session/{token}`	Session info
POST	`/interview-session/{token}/verify`	Email verification
POST	`/interview-session/{token}/start`	Start
POST	`/interview-session/{token}/turn`	Submit an answer
POST	`/interview-session/{token}/complete`	Finish
GET	`/interview-session/{token}/audio/{file}`	TTS audio for a question

Analytics - /analytics

Method	Path	Description
GET	`/analytics/dashboard, /candidates, /funnel`	Headline metrics and funnel
GET	`/analytics/roles, /trend`	Per-role breakdown, time series
GET	`/analytics/report, /export`	Report data and file export
GET	`/analytics/callbacks, /ai-screening`	Callback queue, screening stats

Billing - /api/billing

Method	Path	Description
GET	`/api/billing/status`	Trial / paid status
POST	`/api/billing/create-order`	Create Razorpay order
POST	`/api/billing/verify`	Verify payment signature
POST	`/api/billing/webhook`	Razorpay webhook

Super admin - /api/superadmin

Method	Path	Description
GET	`/api/superadmin/stats, /users`	Platform statistics, all users
GET	`/api/superadmin/requests`	Access requests
POST	`/api/superadmin/requests/{id}/approve | reject`	Approve creates org + admin user
GET/POST	`/api/superadmin/companies`	List / create companies
POST	`/api/superadmin/companies/{id}/grant-access`	Grant free days

Health

Method	Path	Description
GET	`/, /ping, /health`	Liveness checks

10. Frontend

The frontend is a Next.js app-router project. `lib/api.ts` wraps fetch, attaches the JWT from localStorage and the `X-View-As-Org` header when a super admin is impersonating, and clears the session on 401. `lib/auth-context.tsx` holds the logged-in user; `components/billing-gate.tsx` shows the paywall on 402.

Route	Audience	Purpose
`/, /pricing, /contact, /terms, /privacy, /refund-policy`	Public	Marketing and legal pages
`/login, /request-access`	Public	Sign-in and company sign-up
`/dashboard`	Recruiter	Headline metrics and charts
`/jobs`	Recruiter	Job postings
`/candidates, /resumes`	Recruiter	Pipeline table and candidate profiles
`/ai-recruiter`	Recruiter	AI Hiring Assistant configuration per job
`/campaigns`	Recruiter	Bolna calling campaigns
`/interviews, /interviews/[id]`	Recruiter	L1 interview list and report
`/l2-interviews, /l2-interviews/[id]`	Recruiter	L2 interview list and report
`/analytics, /settings`	Recruiter	Analytics and org settings / billing
`/superadmin, /companies, /requests`	Super admin	Platform console
`/interview/[token]`	Candidate	Interview room

10.1 Candidate interview room

- Requests camera and microphone with `getUserMedia` (requires HTTPS) and records each answer with `MediaRecorder`.
- Plays each question's TTS audio, then records until the candidate submits.
- Collects integrity telemetry per answer: `visibilitychange` focus loss (count and ms), paste events, fullscreen exits and time to first answer.
- Posts video, audio, duration and telemetry to `/turn` as multipart form data.

11. Configuration

The backend reads `backend/.env.local` through pydantic-settings; the frontend reads `frontend/.env.local`. `NEXT_PUBLIC_API_URL` is baked in at build time.

Variable	Default	Purpose
`MONGODB_URI / MONGODB_DB`	required / voiceai	Database connection
`SECRET_KEY`	must be set	JWT signing; interview-token HMAC fallback
`OPENAI_API_KEY, OPENAI_STT_MODEL`	required, whisper-1	Parsing, scoring, STT, TTS, evaluation
`OPENROUTER_API_KEY / MODEL / URL`	required	Classification and drafting
`SHORTLIST_THRESHOLD`	70	Resume score needed to shortlist
`BOLNA_API_KEY / AGENT_ID / CALLBACK_API_TOKEN`	-	AI phone screening

Variable	Default	Purpose
`GOOGLE_DRIVE_API_KEY`	-	Drive folder import
`INTERVIEW_PUBLIC_BASE_URL`	http://localhost:3000	Origin used in invite links
`INTERVIEW_TOKEN_TTL_HOURS`	48	Link lifetime
`INTERVIEW_QUESTION_COUNT / MAX_FOLLOWUPS`	8 / 3	Interview length
`INTERVIEW_INTEGRITY_ENABLED`	true	Run integrity analysis
`INTERVIEW_REQUIRE_EMAIL_VERIFY / MAX_VERIFY_ATTEMPTS`	true / 6	Pre-interview email check
`INTERVIEW_TOKEN_SECRET`	falls back to SECRET_KEY	Dedicated token HMAC key
`INTERVIEW_MEDIA_DIR / MEDIA_MAX_MB`	media/interviews / 200	Recording storage
`SMTP_HOST / PORT / USER / PASSWORD / FROM_EMAIL / USE_TLS`	587, TLS on	Outbound email
`RAZORPAY_KEY_ID / KEY_SECRET / WEBHOOK_SECRET`	-	Payments (inert until set)
`TRIAL_DAYS / BILLING_PLAN_AMOUNT_INR / BILLING_PLAN_DAYS`	7 / 2999 / 30	Trial and plan

12. Deployment

Production runs on a single AWS EC2 instance (t3.small or larger, Ubuntu 24.04, ~20 GB gp3, Elastic IP). HTTPS is required because browsers only allow camera access on secure origins. Use separate subdomains for the app and API, since API routes are not under one prefix.

Component	Runs as	Notes
Frontend	systemd `hireonomous-web`: npm start on :3000	Run `next build` first; needs 2 GB RAM or a swapfile
Backend	systemd `hireonomous-api`: uvicorn app.main:app, 2 workers, :8001	WorkingDirectory must be backend/ (media path is relative)
nginx	Reverse proxy + certbot TLS	API: `client_max_body_size 250M`, `proxy_read_timeout 300s`
MongoDB	Atlas	Allow the Elastic IP in Network Access
Recordings	EBS disk at backend/media/interviews	Snapshot the volume, or move storage to S3/R2

```
# install
sudo apt install -y python3.11 python3.11-venv nginx certbot python3-certbot-nginx
cd backend && python3.11 -m venv venv && ./venv/bin/pip install -r requirements.txt
cd ../frontend && npm ci && npm run build

# services and TLS
sudo systemctl enable --now hireonomous-api hireonomous-web
sudo certbot --nginx -d app.example.com -d api.example.com
```

Only ports 22 (restricted), 80 and 443 should be open. After go-live, point the Bolna agent's callback at the public API URL. See `DEPLOYMENT.md` for the full runbook.

13. Known limitations and recommendations

Area	Current state	Recommendation
JWT secret	`core/auth.py` falls back to a hard-coded default if SECRET_KEY is unset	Fail startup when SECRET_KEY is missing
Bolna webhook	`/bolna/webhook` accepts unauthenticated POSTs that update candidate records	Require a shared secret or verify the source IP / signature
CORS	`allow_origins=["*"]`	Restrict to the app origin in production
Recording storage	Local disk on one instance	Move InterviewStorage to S3 / R2; enables horizontal scaling
Background evaluation	`asyncio.create_task` in the API process	Lost on restart; use a job queue, or sweep interviews left in evaluation_status=pending
Billing gate	Fails open on database errors	Intentional for availability; monitor the logged errors
Legacy code	Twilio, Vapi, demo voice and simulation routers are present but not all registered	Remove or isolate unused paths
Maintenance scripts	Ad-hoc scripts in backend/ root (check_db, find_*, test_*)	Move to scripts/ or delete; add an automated test suite
Integrity v2	No face match, gaze or voice fingerprinting	Planned; needs GPU or external vision service